

腾讯地图算法岗复试（第三面）

核心问题与参考答案

面试准备笔记

2026 年 4 月 20 日

前言

本文档基于 CSIG 腾讯地图暑期实习复试的真实业务背景整理：涉及微信小程序网约车业务、传统推荐（上下车点推荐、用户补贴）以及 LLM 落地（AI 客服、端内智能体）。以下问题紧扣面试官可能追问的技术深度、业务理解和落地能力。

1 网约车业务 & 传统推荐

1.1 上下车点推荐

问题1：如何定义上下车点推荐的目标？是 CTR、CVR 还是完单率？

- (1) **核心目标**：完单率（用户接受推荐并成功完成订单）和用户满意度（投诉率、低评分比例）。单纯的点击率（CTR）可能导致“好看不好用”的推荐。
- (2) **辅助指标**：推荐上车点的采纳率、用户手动修改比例、平均步行距离。
- (3) **多目标优化**：可使用加权求和或帕累托优化，例如 $Score = \alpha \cdot \text{完单率} - \beta \cdot \text{投诉率} - \gamma \cdot \text{步行距离}$ 。

问题2：冷启动用户或冷门地点怎么处理？

- (1) **用户冷启动**：基于地理位置（当前定位周边 200m 内的历史热门上车点）、时间上下文（早高峰推荐地铁口，晚高峰推荐小区门口）。
- (2) **地点冷启动**：利用 POI 类别相似性（如“地铁站”类别的上车点分布模式可迁移到新建地铁站）；或采用基于规则的 fallback（如“请走到主路交叉口”）。
- (3) **混合策略**：冷启动阶段使用流行度 + 距离加权，积累少量数据后快速切换为个性化模型。

问题3：你们用了哪些特征？时空特征如何融合？

- (1) **用户特征：**历史常用上车点、打车频次、会员等级。
- (2) **上下文特征：**时间（小时、是否周末）、天气（雨雪天推荐室内或遮雨处）、实时路况。
- (3) **POI 特征：**类别（公交站/小区/商场）、人流量热度、是否有违停风险。
- (4) **融合方式：**将时间离散化为 one-hot 或 embedding，与地理坐标（经度/纬度分桶）拼接后输入双塔模型；或使用 GBDT 自动处理特征交叉。

问题4：如何处理稀疏性问题？比如某个上车点历史订单很少。

- (1) **层次化建模：**先建立“区域→路段→具体点”的层次结构，利用上层统计信息平滑下层。
- (2) **图神经网络：**将上车点作为节点，地理相邻或功能相似的点之间连边，通过图卷积聚合邻居信息。
- (3) **贝叶斯平滑：**对点击率/完单率做 Beta 先验，用全局均值拉低稀疏点的方差。

问题5：设计一个从“用户打开小程序”到“推荐上车点”的完整链路。

- (1) **触发：**用户进入网约车页面，获取实时 GPS 定位。
- (2) **候选召回：**以定位为中心，半径 500m 内所有历史上车点 + 附近 POI 的候选上车区。
- (3) **排序：**使用轻量级 XGBoost 模型，输入特征（距离、历史完单率、时间、天气、用户 embedding）输出得分。
- (4) **重排：**避免推荐过于聚集（如连续推荐同一个点），加入多样性惩罚；同时应用业务规则（如机场强制推荐指定上车区）。
- (5) **展示：**地图上高亮显示 35 个推荐点，按得分排序。

1.2 用户补贴策略

问题1：补贴策略的目标是什么？如何衡量 ROI？

- (1) **目标：**在有限预算下最大化净增订单（或 GMV），同时控制获客成本（CAC）。
- (2) **ROI 计算：**
$$ROI = \frac{\text{补贴带来的增量收益}}{\text{补贴成本}}$$
。增量收益 = (实验组订单额 - 对照组订单额) - 自然增长。
- (3) **辅助指标：**补贴核销率、用户次周留存、自然单占比。

问题2：如何决定给哪个用户发券、发多少？

- (1) **用户分群**：基于历史行为分为价格敏感型、流失边缘型、忠诚型。只对前两类投放。
- (2) **模型决策**：可使用 Uplift Model 预测“发券 vs 不发券”的增量订单概率；或采用响应模型 + 成本约束的线性规划。
- (3) **券额策略**：通常采用离散值（2 元、5 元、8 元），通过离线评估选择性价比最高的档位；也可用强化学习动态调额。

问题3：有没有考虑过 Uplift Model？如何区分自然转化和补贴增量？

- (1) **模型选择**：可使用 Two-Model（分别建模实验组和对照组）、Class Transformation（将增量作为目标）或直接使用因果森林。
- (2) **数据获取**：需要随机 A/B 实验，一部分用户发券（treatment），一部分不发（control），收集订单结果。
- (3) **评估**：Qini 曲线或 AUUC 指标；最终只对 uplift 分数高的用户发券，避免浪费预算在“不发券也会打车”的用户上。

问题4：预算有限时，如何分配不同用户群的补贴？

- (1) **背包思想**：将每个用户（或用户群）看作一个物品，其“补贴金额”为成本，“增量订单概率”为价值，求解总预算约束下的价值最大化。
- (2) **在线分配**：使用 PID 控制器或自适应调价算法，实时调整不同时段的补贴强度。
- (3) **业务优先级**：高价值但近期沉默的用户可给予更高预算上限。

2 LLM 落地

2.1 AI 客服（退款、问责等）

问题1：用户投诉“司机绕路，要求退款”，AI 客服如何处理？需要哪些模块？

- (1) **意图识别**：分类为“退款申请 - 绕路”。
- (2) **信息抽取**：从对话中提取订单号、投诉金额、绕路描述（可选）。
- (3) **决策逻辑**
 - 查询订单实际轨迹和预估路线，计算绕路比例。
 - 若绕路比例 $\geq 20\%$ 且金额较小（ ≤ 20 元），自动触发退款。
 - 否则标记为需人工审核。
- (4) **回复生成**：基于预置模板 + 具体退款金额，通过 LLM 润色语气。
- (5) **安全护栏**：退款操作需二次确认，且最终金额不能超过订单原价。

问题2：如何保证 AI 客服的安全性和合规性？避免承诺无法实现的退款？

- (1) **规则覆盖：**关键操作（退款、改签）必须走硬编码的业务逻辑，LLM 仅负责自然语言理解与生成，不直接输出操作指令。
- (2) **输出校验：**LLM 生成的退款金额需与后台计算值做匹配，不一致则拒绝并转人工。
- (3) **拒绝回答：**对“如何刷单”“绕过平台规则”等恶意提问，直接返回标准拒绝话术。
- (4) **人工兜底：**任何涉及高额（≥50 元）或复杂责任认定的对话自动转人工。

问题3：遇到情绪激动、辱骂或重复问题，如何设计对话策略？

- (1) **情绪识别：**使用轻量级分类器（或 LLM 做 few-shot）检测愤怒、辱骂关键词。
- (2) **策略分流：**
 - 轻度负面：先发送安抚话术（“很抱歉给您带来不便，我们正在全力处理”）。
 - 重度辱骂：直接转人工，并保留对话记录。
 - 重复问题：检测到连续 3 轮相同意图，自动升级或提供“转人工”快捷按钮。
- (3) **记录分析：**统计高频投诉点，驱动产品改进。

问题4：AI 客服如何与人工客服协作？转人工的触发条件是什么？

- (1) **触发条件：**
 - 用户明确要求“转人工”。
 - AI 连续 2 次无法理解用户意图（置信度 ≤ 0.3）。
 - 涉及高风险操作（大额退款、账号封禁申诉）。
 - 用户情绪评分低于阈值。
- (2) **协作方式：**转人工时携带对话历史、已确认的订单信息、AI 的初步结论，让人工快速接手。
- (3) **闭环学习：**人工处理结束后，将案例回灌到 AI 训练集中，持续提升自助解决率。

问题5：如何评估 AI 客服的效果？

- (1) **核心指标：**自助解决率（无需人工介入的比例）、转人工率、用户满意度（CSAT）。
- (2) **效率指标：**平均首次响应时间、平均解决时长。
- (3) **风险指标：**错误退款率（不应退款却退款的）、合规事故数。
- (4) **长期价值：**用户投诉后 7 日内留存率变化。

2.2 APP 端内智能体

问题1：端内智能体可以做什么？设计一个具体的交互流程。

- (1) **典型场景**：用户语音输入“帮我叫一辆从公司到机场的车，尽量走高速”。
- (2) **流程**：
 - (a) ASR 转文本。
 - (b) LLM 解析意图（叫车）+ 参数（起点=公司地址，终点=机场，偏好=高速优先）。
 - (c) 调用地图 API：获取公司当前位置，规划高速路线，估算价格。
 - (d) 确认对话：“为您找到路线，预计 45 分钟，费用 80 元，是否确认叫车？”
 - (e) 用户确认后调用网约车下单接口，返回订单信息。

问题2：如何让 LLM 理解地图特有的操作？比如“避开拥堵路段”“收藏这个地点”。

- (1) **Function Call 机制**：预定义一批地图 API 函数（`search_poi`, `plan_route`, `add_favorite`, `avoid_congestion` 等），LLM 输出结构化函数调用。
- (2) **上下文注入**：在 system prompt 中明确说明可用函数及其参数格式，并提供 few-shot 示例。
- (3) **实体识别**：对“这里”“那个停车场”等指代，通过对话状态追踪模块解析为具体 POI ID。

问题3：端侧部署 LLM 的挑战与对策。

- (1) **挑战**：模型大小（端侧通常 $\leq 1\text{B}$ 参数）、推理速度（需 $\leq 200\text{ms}$ ）、内存占用、隐私合规。
- (2) **对策**：
 - 模型量化（INT8/INT4），或使用蒸馏后的小模型（如 MobileLLM）。
 - 简单意图端侧处理，复杂推理（多轮对话、路线规划）走云端大模型。
 - 敏感信息（如用户常去地点）仅留在端侧，不上云。

问题4：如何评估智能体的任务完成率？

- (1) **自动评估**：对于明确指令（如“设置家地址”），检查 API 是否成功调用且参数正确。
- (2) **人工评估**：随机采样对话，标注“任务成功/失败/部分成功”。
- (3) **用户反馈**：在对话结束时让用户点踩/点赞，或分析后续行为（如用户是否手动修改了智能体的操作结果）。
- (4) **效率指标**：完成同一任务所需的平均对话轮数。

3 综合 & 软技能

问题1：你为什么选择腾讯地图而不是高德/百度？腾讯地图的竞争优势？

- 生态协同：微信小程序内可直接调用地图，无需跳转 APP，社交裂变成本低。
- 数据优势：腾讯系产品（微信、QQ、滴滴）积累的海量位置和出行数据。
- 技术侧重：CSIG 注重产业互联网，地图与云、AI 结合紧密（如智慧交通、车联网）。

问题2：你做的上下车点推荐，如果现在重新做，会有什么改进？

- 引入实时动态信息：比如当前上车点是否有交警、临时施工，通过众包或路况 API 获取。
- 多模态输入：用户拍照上传上车点照片，由多模态模型判断是否安全、好找。
- 强化学习：将推荐视为序列决策，以长期用户留存为奖励。

问题3：你对 LLM 在出行领域的未来应用有什么看法？

- 超个性化行程管家：LLM 记住用户偏好（偏爱咖啡店、避开某条路），主动推荐“下班后去健身房的最优路线”。
- 自然语言数据挖掘：分析用户评价中的非结构化文本，发现地图 POI 的隐藏属性（“这家店门口常堵车”）。
- 多模态交互：用户对着街景说“在这里打车”，模型识别位置并直接下单。

4 复试冲刺建议

1. 画架构图：提前画出“上下车点推荐系统”和“AI 客服”模块流程图，面试时可直接展示。
2. 准备三个“最亮点”故事：推荐优化案例、LLM 落地坑与解法、跨团队协作经历。
3. 准备反问问题：
 - “目前 AI 客服的自助解决率是多少？团队的下一个优化目标？”
 - “上下车点推荐是否考虑过结合实时司机位置？”
 - “端内智能体会探索多模态（用户发照片找地点）吗？”
4. 模拟面试：限时回答，注重逻辑清晰、先结论后细节，主动说明方案的局限性和风险控制。

5 如何回答“更青睐于 to B 还是 to C”

在推荐系统岗位的面试中，这个问题考察的是你对业务本质的理解、对不同场景推荐系统差异的认知，以及与当前岗位的匹配度。建议采用“偏向 to C + 保留开放性”的应答策略。

推荐回答框架

- (1) **承认两者共通性**：to C 和 to B 在召回排序、特征工程、AB 实验等底层技术上相通。
- (2) **表达当前偏好 (to C)**：结合个人经历和兴趣，说明倾向 to C 的三个理由。
- (3) **强调未完全确定**：表明愿意根据业务需求学习成长，对 to B 持开放态度。
- (4) **拉回当前岗位**：表达对腾讯地图网约车推荐业务的热情和匹配度。

参考话术（可灵活调整）

“我认为 to C 和 to B 的推荐系统在底层技术上有很多共通之处，比如召回排序、特征工程、AB 实验框架都是相通的。不过从我个人的技术兴趣和项目经历来看，我目前更倾向于 to C 推荐。原因有三：第一，C 端用户规模大、行为丰富，适合尝试大规模深度学习、在线学习等技术，挑战性和成长空间更大；第二，C 端推荐直接面向普通用户，效果提升能立刻改善用户体验，比如上下车点推荐减少步行距离，这让我很有成就感；第三，我过往的项目主要集中在 C 端场景，对 CTR/CVR 预估、冷启动、多目标优化比较熟悉，上手会更快。

当然，我并没有完全确定自己只做 to C。我觉得优秀的算法工程师不应该给自己设限。to B 推荐在因果推断、小样本学习、可解释性等方面同样有很多有意思的问题，如果未来业务需要，我很愿意去学习和探索。只是就目前这个阶段，我的兴趣和优势更集中在 C 端。

回到咱们腾讯地图的岗位，我了解到当前重点是网约车场景的上下车点推荐和用户补贴，这恰恰是我最想深入的方向。我会把全部精力投入进去，确保上线效果。未来如果团队需要拓展 to B 的智能体服务，我也会积极学习。”

需要避免的说法

- “我只做 C 端，B 端没意思。” —— 显得狭隘，缺乏适应性。
- “B 端更高级，C 端太简单。” —— 显得傲慢且不客观。
- “我没想过这个问题，都行吧。” —— 缺乏思考深度。
- “我更喜欢 B 端，但为了面试就说 C 端。” —— 显得虚伪。

一句话总结

表达对 to C 的倾向（基于经历和兴趣），同时留出成长空间（愿意学习 to B），最后落脚到对当前岗位的热情。 这样既展示了个性，又体现了务实和开放的心态。